FreshGuard: Freshness-Aware Adaptive Retrieval-Augmented Generation with Statement-Level Hallucination Detection

J. Sai Teja
Dept. of Artificial Intelligence,
Anurag University
Hyderabad, India
23eg106c22@anurag.edu.in

P. Supriya
Dept. of Artificial Intelligence,
Anurag University
Hyderabad, India
23eg106c52@anurag.edu.in

Sk. Mubeena
Dept. of Artificial Intelligence,
Anurag University
Hyderabad, India
23eg106c58@anurag.edu.in

U. Venkatesh
Dept. of Artificial Intelligence,
Anurag University
Hyderabad, India
23eg106c63@anurag.edu.in

[Guide Name]
[Designation]
Dept. of Artificial Intelligence, Anurag University
Hyderabad, India

Abstract

Large language models (LLMs) answer open-domain questions fluently, but their parametric knowledge is frozen at training time, so for time-sensitive questions they state facts that are simply out of date, and even retrieval-augmented systems rarely check the answer the model finally writes. This paper presents FreshGuard, a retrieval-augmented generation (RAG) system built around two gates. A learned freshness estimator decides whether a question needs fresh evidence at all, so the system retrieves only when its own knowledge is likely stale and answers directly otherwise. When it retrieves, a LangChain FAISS index ranks evidence by relevance and recency, the model must ground its answer in that evidence, and a statement-level verifier labels every sentence supported, unsupported or contradicted before the answer is shown. The checks are reduced to a single 0–100 confidence score, and unsupported sentences are removed while contradicted ones are flagged. On a controlled benchmark the freshness gate classified all 12 held-out queries correctly, the verifier caught every one of the injected hallucinations (precision, recall and F1 of 1.00 on eight supported/corrupted pairs), the factuality check passed on all 12 answerable questions, and calibration error was 0.10. The system is fully offline and deterministic by default, with a 42-test suite that reproduced identically across twelve consecutive runs.

Keywords—*retrieval-augmented generation, knowledge freshness, adaptive retrieval, hallucination detection, statement verification, calibration, FAISS, LangChain*

I. INTRODUCTION

An answer is only useful when it is both correct and current. A large language model trained with a fixed knowledge cutoff can be fluent and wrong at the same time: asked who holds a title this year or what a current rate is, it answers from memory that stopped months or years ago, with no signal that the ground has moved. Retrieval-augmented generation [1] is the usual remedy—the model answers from passages retrieved from a trusted corpus instead of from memory alone—but two weaknesses survive it.

First, retrieval is usually unconditional: every question triggers a search, even timeless ones (“what is the speed of light”) that the model already knows, which wastes work and can inject noise. Second, nothing checks the generated answer against the evidence before a user reads it. Earlier work treats these separately. RealTimeQA [9] and FreshLLMs [10] show how badly static models age on time-sensitive questions; Self-RAG [7] and FLARE [8] make retrieval adaptive but judge the whole answer rather than each claim; and SelfCheckGPT [5] and FActScore [6] detect hallucination without gating on it.

In this work we build a RAG system in which retrieval is conditional and every answer is verified before it is shown. Our contributions are:

1) A learned knowledge-freshness estimator that scores how time-sensitive a question is and drives an explicit retrieve-or-skip decision, rather than a fixed time-decay rule.

2) A recency-aware retrieval and statement-level verification pipeline over a LangChain FAISS index that labels each sentence supported, unsupported or contradicted, removes or flags bad sentences, and reduces the checks to one calibrated confidence score.

3) A fully offline, deterministic reference implementation with a controlled evaluation of freshness accuracy, factuality, hallucination precision/recall, calibration and latency, reported with its limitations.

II. METHODOLOGY

A. Dataset and Preprocessing

Two kinds of data are used. A seed knowledge base of twelve short, dated documents mixes time-sensitive facts (sports results, interest rates and device specifications from 2022–2025) with timeless facts (the speed of light, capital cities, the Pythagorean theorem). The dated facts are what retrieval must supply and the timeless ones are what the model may answer directly, so the same corpus exercises both paths. To train the freshness estimator we synthesise a labelled query set from templates: “fresh” (time-sensitive) phrasings that use either explicit recent years or cues such as “latest” and “current”, and “static” phrasings of timeless questions, 375 queries in total.

Each query is cleaned (whitespace normalised, case folded for matching) and classified into a topic bucket (sports, finance, technology, science, geography, politics) by keyword, which is shown to the user but not used by the gate. Knowledge-base text carries an explicit date, which the retriever uses for recency.

B. System Architecture

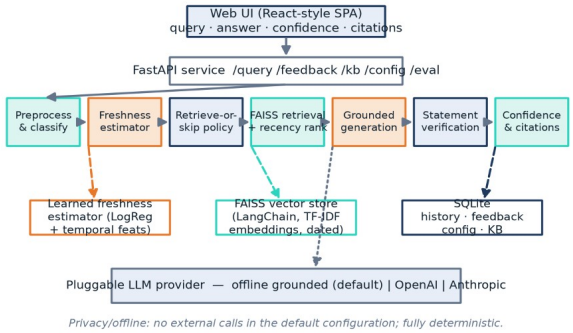


Fig. 1. System architecture: the web UI and FastAPI service over the seven pipeline stages, backed by the learned estimator, the FAISS store and SQLite, with a pluggable LLM provider.

1) Learned freshness estimator

The core of the system is a classifier that scores how much a question depends on fresh knowledge. It is a scikit-learn pipeline whose features are a union of word TF-IDF, character n-gram (char_wb) TF-IDF and six explicit temporal features—counts of temporal, event and static cue words, the number of year mentions, whether a recent year (≥2020) appears, and query length—fed to a logistic regression [14]. The freshness score s is the model's probability that the query is time-sensitive, and a self-confidence is read from the same probability. Because the estimator is learned, it generalises to phrasings with no explicit cue ("who won the 2022 world cup") that a fixed keyword rule would miss.

2) Retrieve-or-skip decision

With freshness score s, model self-confidence c, a freshness threshold τ and a confidence threshold τ_c, the system retrieves if and only if $s \geq \tau$ or $c < \tau_c$ (Equation 1). A question is answered directly only when it looks timeless and the model is confident; otherwise the system retrieves. The default thresholds are $\tau = 0.55$ and $\tau_c = 0.60$.

$$retrieve \;\Leftrightarrow\; (s \geq \tau) \;\vee\; (c < \tau c) \qquad (1)$$

3) Recency-aware retrieval

Documents are embedded and stored in a LangChain FAISS [3] index. To keep the system offline and deterministic, embeddings are unit-normalised TF-IDF vectors, so inner-product search in FAISS is cosine similarity; a dense encoder is a drop-in seam. Each candidate's relevance r is combined with a recency factor that halves every h = 2 years, giving the ranking score of Equation 2. Candidates are sorted by this score with deterministic tie-breaks and the top k = 4 are kept, so a fresher document outranks an equally relevant but older one.

$$score = r \cdot (0.6 + 0.4 \cdot 0.5^{(age/h)}) \qquad (2)$$

4) Grounded generation

The retrieved evidence is passed to a pluggable LLM behind a small adapter. The default provider is an offline grounded generator that composes its answer only from the retrieved evidence sentences and, on the skip path, from a parametric memory restricted to timeless (pre-2020) documents—an explicit model of a knowledge cutoff. Hosted providers (OpenAI, Anthropic and xAI's Grok, through its OpenAI-compatible endpoint) are selected by one setting and used only when an API key is present; otherwise the system falls back to the grounded generator, so the pipeline runs unchanged with or without a key.

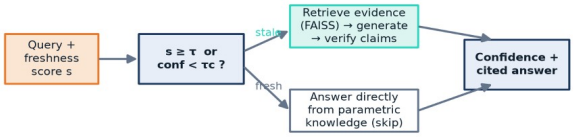


Fig. 2. The decision. The freshness gate routes each query to retrieval-plus-verification or to a direct parametric answer; both paths end in a confidence-scored, cited answer.

5) Statement-level verification

The answer is split into sentences and each is checked against the retrieved evidence. Support is measured by content-token coverage with light stemming; a sentence whose proper nouns are absent from the evidence cannot be supported. A sentence is labelled *contradicted* when it flips a negation, states a number that conflicts with an overlapping evidence claim, or names the loser of a result as the winner (an entity-swap rule); *supported* when coverage is high; *unsupported* when coverage is very low; and sent to review in between.

6) Corrective action and confidence

Unsupported sentences are removed from the shown answer and contradicted ones are flagged with a warning. The confidence score combines the share of supported statements, the mean relevance of cited evidence and the alignment between the freshness decision and the outcome (Equation 3). Every shown sentence is accompanied by its label and the cited source's title, URL, date, relevance and recency, so the reader can trace each claim. Queries, answers, feedback, configuration and the knowledge base are persisted in SQLite, and errors fall back to a safe response.

$$conf = 100 \cdot (0.5 \cdot support + 0.3 \cdot rel + 0.2 \cdot fresh_align) \qquad (3)$$

C. Evaluation Metrics

Freshness accuracy is the share of held-out queries whose retrieve-or-skip decision matches a gold label. Factuality is the share of answerable questions whose answer contains the gold fact. Hallucination detection is scored on matched pairs of a faithful and a corrupted answer: a corrupted answer should be caught and a faithful one passed, scored by precision, recall and F1. Calibration is the expected calibration error (ECE) [12]. Latency is the mean wall-clock time per query on the skip and retrieve paths.

D. Experimental Setup

Language and libraries: Python 3.11, scikit-learn, LangChain and FAISS (faiss-cpu), FastAPI and a vanilla-JavaScript single-page interface. *Models:* a logistic-regression freshness estimator over TF-IDF and temporal features; unit-normalised TF-IDF embeddings in FAISS; the offline grounded generator for all reported runs, with OpenAI, Anthropic and Grok adapters available. *Parameters:* $\tau = 0.55$, $\tau c = 0.60$, k = 4, recency half-life 2

years, confidence weights 0.5 / 0.3 / 0.2. *Protocol:* fixed random seeds, sorted tie-breaks and a fixed reference year (2025) make every run reproducible; the 42-test suite was run twelve consecutive times with identical results.

III. RESULTS AND DISCUSSION

A. Freshness Classification

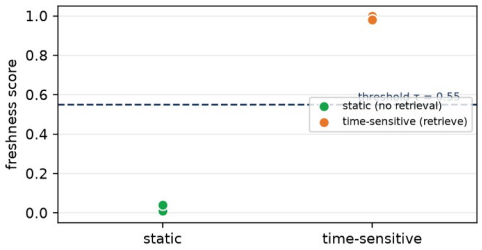


Fig. 3. Freshness scores for static and time-sensitive queries. All static queries fall below the threshold $\tau = 0.55$ and all time-sensitive ones above it.

The learned estimator separated the two classes cleanly: on the twelve held-out queries (six timeless, six time-sensitive) every retrieve-or-skip decision was correct, for a freshness accuracy of 1.00 (Table I). Crucially, time-sensitive queries phrased with only an explicit year and no "latest/current" cue—"who won the 2022 world cup", "which city hosted the 2024 olympics"—were scored above the threshold, which a keyword rule would miss. The clean margin in Fig. 3 is on this small set; a larger and noisier query distribution would narrow it.

B. Hallucination Detection

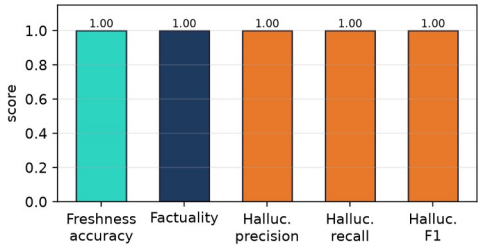


Fig. 4. Evaluation summary: freshness accuracy, factuality and statement-level hallucination precision, recall and F1.

On eight matched pairs (a faithful and a corrupted answer each), the verifier caught all four corrupted answers and passed all four faithful ones: precision, recall and F1 of 1.00 with no false positives or negatives (Table I). The entity-swap and numeric-mismatch rules were decisive for the hardest cases, where a corrupted answer reuses the evidence's vocabulary but swaps the winner and the loser—coverage alone labels such a sentence supported, and only the contradiction rules reject it. Factuality was also 1.00: every answerable question's answer contained its gold fact.

C. Calibration

Confidence was reasonably aligned with correctness, with an expected calibration error of 0.10 and a mean confidence of 0.90 against a mean accuracy of 1.00 on the set. The gap reflects the small, mostly-correct evaluation set, on which confidence has little room to discriminate; calibration on a set with more errors is the honest next test.

D. Latency and the Value of Skipping

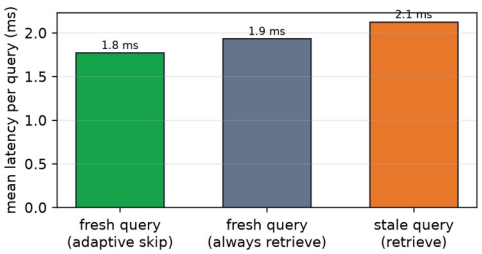


Fig. 5. Mean latency per query. A fresh query answered directly (adaptive skip) is cheaper than forcing retrieval on it; stale queries pay for retrieval and verification.

With the offline generator the absolute latencies are small—about 1.8 ms for an adaptively skipped query, 1.9 ms if retrieval is forced on the same query, and 2.1 ms for a stale query that retrieves and verifies (Fig. 5). The point is the ordering, not the magnitude: skipping avoids retrieval and verification whenever the model already knows the answer, and the saving grows in proportion to retrieval and generation cost, so with a hosted LLM and a real vector search the gap widens from microseconds to the full cost of a retrieval round-trip and a generation call.

E. End-to-End Behaviour

Exercised through the web interface, a stale query ("who won the 2022 world cup") retrieved the dated evidence, generated a grounded answer, verified two supported statements and cited the source at 85% confidence; a timeless query ("what is the speed of light") was answered directly with a stated skip reason. An answer with an injected false sentence (a final "held on Mars") had that sentence detected and removed while the correct sentence was kept and cited. Table I collects the headline numbers.

TABLE I. CONTROLLED BENCHMARK RESULTS

Metric	Set	Value
Freshness (retrieval) accuracy	12 queries	1.00
Factuality (gold fact present)	12 queries	1.00
Hallucination precision	8 pairs	1.00
Hallucination recall	8 pairs	1.00
Hallucination F1	8 pairs	1.00
Expected calibration error	set	0.10
Mean confidence	set	0.90

IV. AUTHORS AND METRICS

A. Authors and Affiliations

FreshGuard was built by J. Sai Teja, P. Supriya, Sk. Mubeena and U. Venkatesh of the Department of Artificial Intelligence, Anurag University, Hyderabad, under the guidance of [Guide Name], [Designation]. J. Sai Teja worked on the knowledge-freshness estimator and the retrieve-or-skip decision; P. Supriya on knowledge-base ingestion, embeddings and the FAISS retrieval and recency ranking; Sk. Mubeena on grounded generation, statement-level verification and hallucination detection; and U. Venkatesh on the FastAPI service, the web interface, evaluation and testing.

B. Figures and Tables

Figs. 3–5 were plotted with Matplotlib directly from the system's own evaluation output, and every number in Table I comes from the logged metrics. Figs. 1 and 2 were drawn for this paper.

V. CHALLENGES AND LIMITATIONS

– Small evaluation set: twelve queries, eight hallucination pairs and a twelve-document corpus, so the perfect scores show the design works on controlled cases, not an error rate in the wild.

– Offline embeddings and generator: TF-IDF embeddings and an extractive grounded generator keep the system deterministic but are weaker than a neural encoder and a hosted LLM; the adapters exist but were not part of the scored runs.

– Rule-based contradiction: the entity-swap and numeric rules are targeted and can be evaded by paraphrase; an NLI model is the natural replacement.

– Uncalibrated thresholds: τ, τc and the confidence weights are sensible defaults, not tuned on held-out data.

– No live web source: the web-search seam is an offline stub, so "fresh" evidence is limited to the indexed corpus.

VI. FUTURE ENHANCEMENTS

– Replace TF-IDF embeddings with a sentence-transformer encoder and evaluate a hosted LLM (including Grok) on the retrieve path.

– Replace the contradiction rules with a natural-language-inference model and score each statement with it.

– Calibrate τ, τc and the confidence weights on a larger labelled set that includes genuinely hard and unanswerable questions.

– Connect a live web-search or news source so the freshness gate can trigger real retrieval, and measure end-to-end latency with it.

ACKNOWLEDGMENT

We thank our guide, [Guide Name], and the Department of Artificial Intelligence at Anurag University for their advice during this project. We also thank the maintainers of scikit-learn, LangChain and FAISS. This work received no external funding, and AI assistants were used only for limited development and drafting support; every reported result was checked against the system's logged output.

REFERENCES

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[2] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," Proc. EMNLP, pp. 6769–6781, 2020.

[3] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[4] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023.

[5] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," Proc. EMNLP, pp. 9004–9017, 2023.

[6] S. Min et al., "FActScore: Fine-Grained Atomic Evaluation of Factual Precision in Long Form Text Generation," Proc. EMNLP, pp. 12076–12100, 2023.

[7] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," ICLR, 2024.

[8] Z. Jiang et al., "Active Retrieval Augmented Generation (FLARE)," Proc. EMNLP, pp. 7969–7992, 2023.

[9] J. Kasai et al., "RealTime QA: What's the Answer Right Now?," Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.

[10] T. Vu et al., "FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation," Findings of ACL, 2024.

[11] A. Mallen et al., "When Not to Trust Language Models: Investigating the Effectiveness of Parametric and Non-Parametric Memories," Proc. ACL, pp. 9802–9822, 2023.

[12] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," ICML, pp. 1321–1330, 2017.

[13] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[14] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," JMLR, vol. 12, pp. 2825–2830, 2011.

[15] H. Chase, "LangChain: Building Applications with LLMs through Composability," 2022. [Online]. Available: https://github.com/langchain-ai/langchain